Solo model versus delegated team: reproduce the comparison fairly

Researched 2026-10-12.

What the video says

Muse reports three tasks: a physics game, a JavaScript animation and a bug hunt. The creator reports a 10-2 final score for solo Opus and better team cost on the bug hunt. Treat the score, bug counts, percentages and timings as reported results, not independently measured facts.

The confounders

The two arms differ in both model topology and prompting: the team uses a creator-distributed `/delegate` skill. A model judging a bug report is not proof of a bug; confirmation needs a reproducible failure or a failing test. Creative quality needs a rubric and ideally blinded review. Separate model quality from supervision, task partitioning, effort settings and number of attempts.

Documented mechanism

Claude Code subagents (https://code.claude.com/docs/en/sub-agents) have their own context, tools and configuration, but their requests count against shared usage limits. This makes coordination and duplicate context part of the experiment's cost. The official mechanism does not verify this creator's particular private skill or results.

Reproduction protocol

1. Freeze a starter repository, dependency versions, task prompt and acceptance tests.
2. Compare solo and manager/worker runs under the same wall-clock and spending caps. Include at least three runs per arm with fresh working copies.
3. Record all model usage, worker startup, review rounds, failed attempts and integration time.
4. For bugs, require location, trigger, expected/observed behavior and a regression test. Count false positives separately.
5. For design, score task completion, accessibility and visual coherence before looking at the arm label.
6. Report cost per accepted outcome, median and spread; preserve the raw logs privately and publish a sanitized summary.

Practical next step

Use a small bug-fix benchmark before a broad "build an app" challenge. Do not buy a community subscription merely to reproduce a claim whose task artifacts are unavailable. Define a transparent manager/worker protocol yourself and label it a new experiment rather than the creator's exact implementation.

Outcome

A repeatable benchmark may favor different architectures for different tasks. No universal "always delegate" or "always solo" conclusion follows from three heterogeneous demos.

Reusable project prompt

```
Build Solo-versus-team benchmark. Isolated repeated coding tasks with test-confirmed defects and complete usage accounting. Read the linked official docs first. Use fixture data; keep credentials outside source control. Deliver tests, setup instructions, an evidence/limitations note and a small reproducible example. Do not claim a measurement you did not run. Milestones: Freeze tasks and acceptance tests; Implement both experiment arms; Track retries and review cost; Score blinded outcomes and variance.
```

Delivery and next step

The linked current sources were retrieved for this guide; the original dossier and its extraction caveats are retained in Sources. The guide and printable PDF are research deliverables. The proposed project has not been built. Start with the first milestone and verify its acceptance criteria before expanding the scope.

Original sources

- https://youtu.be/Ug3x37N6Z_E